Week 3, Day 7 (Weekend): System Design Introduction, URL Shortener and Week 3 Review

Prepared for Upendra Kumar Sharma | 3.5 to 4 hours | Main topics: the high-level design (HLD) framework, estimation, building blocks, URL shortener design, Week 3 review and mock

Today's Schedule

- **0:00 to 0:45** Study the HLD framework, estimation and the building blocks (sections 1 to 3).
- **0:45 to 1:15** Read the URL shortener walkthrough (section 4).
- **1:15 to 2:00 Practice:** close this guide, take a blank page, and design the URL shortener aloud in 35 minutes. Then compare with section 4 and note what you missed (section 5).
- **2:00 to 2:45** Take the Week 3 mock interview (section 6).
- **2:45 to 3:15** Check your mini project and write your Week 3 review (sections 7 and 8).
- **3:15 to 3:30** Preview Week 4 (section 9).

1. The HLD Framework

System design interviews test **how you think**, not whether you recall a perfect architecture. Use the same sequence every time, and **speak while you work** so the interviewer can follow.

Time-boxed steps for a 45-minute interview

1. **Clarify requirements (5 min).** Ask questions before drawing anything.
   - **Functional:** what must the system do? Which features are in and out of scope?
   - **Non-functional:** expected scale (users, requests per second, data size), latency target, availability, consistency needs, durability, and any security or compliance needs.
2. **Estimate scale (5 min).** Requests per second, storage, bandwidth, and cache size. Round numbers. The goal is the order of magnitude, since it decides your design.
3. **Define the API (3 to 5 min).** Main endpoints, inputs and outputs, status codes.
4. **Design the data model and pick storage (5 min).** Entities, access patterns, SQL or NoSQL and why.

5. **Draw the high-level architecture (10 min).** Clients, load balancer, services, cache, database, queue, and how a read and a write flow through them.
6. **Deep dive (10 min).** Pick the one or two hardest parts, such as ID generation, hot keys or consistency, and go deep. Let the interviewer steer.
7. **Bottlenecks, failures and trade-offs (5 min).** What breaks first? What if a node or a region fails? How do you scale it 10 times? How do you monitor it?

Habits that score well

- **State your assumptions** and write them down.
- **Start simple, then scale.** A working single-region design first, then the improvements the numbers justify.
- **Justify every choice with a number or a trade-off.** "Redis, because 10,000 reads per second at low latency would otherwise hit the database" beats "Redis, because it is fast".
- **Name the failure modes** and what the user sees when they happen.
- **Do not over-engineer.** Do not add Kafka or sharding unless the requirements need them.

2. Back-of-the-Envelope Estimation

Handy numbers (approximate, for orders of magnitude)

- One day is about 86,400 seconds, call it **100,000**. One month is about **2.5 million** seconds.
- **1 million requests per day is about 12 requests per second.** 100 million per day is about 1,200 per second.
- Peak traffic is commonly **2 to 3 times** the average. Use it to size capacity.
- Sizes: 1 KB = 1,000 bytes, 1 MB = 1,000 KB, 1 GB = 1,000 MB, 1 TB = 1,000 GB.
- **Availability:** 99.9 percent allows about 8.8 hours of downtime a year. 99.99 percent allows about 53 minutes.
- **Latency orders of magnitude:** memory access in nanoseconds (about 100 ns), SSD read in about 100 microseconds, a round trip within a data center about 0.5 ms, a disk seek about 10 ms, a cross-continent round trip about 100 ms or more.
- **Rough capacity per node (hardware and workload dependent):** a relational database handles thousands to tens of thousands of simple queries per second, Redis around 100,000 operations per second, and a single Kafka partition tens of MB per second. Use them to decide how many nodes you need, and say that they are rough.

The usual estimation steps

1. Daily or monthly active users, and actions per user.
2. Writes per second and reads per second (and the read to write ratio).
3. Storage: items per year multiplied by item size multiplied by years of retention.
4. Bandwidth: requests per second multiplied by payload size.
5. Cache: the **hot data share** (often 20 percent of items serve 80 percent of traffic) multiplied by size.

3. Building Blocks Reference

- **Load balancer:** spreads traffic across instances and removes unhealthy ones. Layer 4 (TCP) is faster, layer 7 (HTTP) can route on paths and headers. Algorithms: round robin, least connections, consistent hashing.
- **CDN:** caches static or popular content near users, which cuts latency and origin load.
- **Cache (Redis, Memcached): cache-aside** (the application loads on a miss), **write-through** (write to cache and database together), **write-back** (write to cache first, flush later, which is faster but can lose data). Eviction: LRU or LFU. Watch for **stampedes** (many misses for one hot key at once), stale data and invalidation.
- **Database:**
  - **SQL** for relationships, transactions and flexible queries. **NoSQL** (key-value, document, wide-column) for huge scale and simple access patterns.
  - **Replication:** a leader takes writes and followers serve reads, with **replication lag** (reads can be stale).
  - **Sharding:** split data across nodes by hash (even spread, hard range queries), by range (easy range queries, risk of hot spots) or by a lookup directory. **Consistent hashing** limits data movement when nodes are added.
- **Message queue or log (Kafka, SQS):** decouples producers and consumers, absorbs spikes, enables asynchronous work and fan-out.
- **Object storage (S3):** large files such as images, videos and backups.
- **Search index (Elasticsearch):** full-text and complex filtering.
- **Rate limiter:** protects against abuse and overload (token bucket, sliding window). Week 4 covers it in detail.
- **ID generators:** database sequences, UUIDs, Snowflake-style IDs, or range allocation.
- **Coordination (ZooKeeper, etcd):** leader election, configuration, locks.

Consistency in one paragraph: during a network partition a distributed system must choose between **consistency** (refuse some requests to stay correct) and **availability** (answer, possibly with stale data). Most large systems choose availability with eventual

consistency for reads such as feeds and counts, and stronger consistency for money and inventory. Ask which parts need which.

4. Worked Design: URL Shortener

The prompt: design a service like bit.ly. Users submit a long URL and get a short one. Visiting the short URL redirects to the original.

Step 1: Requirements

- **Functional:** create a short URL from a long URL, optionally with a custom alias and an expiry. Redirect from the short URL to the original. Optionally, click analytics and deleting a link.
- **Non-functional: very read heavy**, low-latency redirects (under about 100 ms), **high availability** (a broken redirect is a broken link on someone's page), durable mappings (links must not be lost), short links that are not trivially guessable if links are private.
- **Out of scope for now:** user accounts, detailed analytics dashboards. State this.

Step 2: Estimation (assumptions)

- 100 million new URLs per month, and a read to write ratio of 100 to 1.
- **Writes:** 100 million divided by 2.5 million seconds is about **40 per second**.
- **Reads (redirects):** 10 billion per month, about **4,000 per second** on average and about **10,000 per second** at peak.
- **Storage:** 100 million per month for 5 years is 6 billion records. At about 500 bytes each, that is roughly **3 TB**.
- **Cache:** about 330 million redirects a day. If 20 percent of links are hot and each is 500 bytes, about **30 GB**, which fits in a small Redis cluster.
- **Key length:** base62 (0 to 9, a to z, A to Z) with 6 characters gives about 57 billion combinations, and **7 characters gives about 3.5 trillion**. For 6 billion links, **7 characters** is comfortable.

Step 3: API

- **POST /api/v1/urls** with a body containing longUrl, and optional customAlias and expiresAt. Returns **201** with the short URL. Returns **409** if a custom alias is taken, and **400** for an invalid URL.
- **GET /{code}** returns a **redirect** (302 or 301, see below) to the long URL. Returns **404** for an unknown code and **410 Gone** for an expired one.
- **DELETE /api/v1/urls/{code}** for the owner. **GET /api/v1/urls/{code}/stats** for analytics.

301 or 302?

- **301 (permanent):** browsers and proxies cache it, so repeat clicks never reach you. That reduces load, but you lose click counts and cannot change or expire the link for those clients.
- **302 (temporary):** every click comes to your server, so you can count it, expire the link and change its target, at the price of more traffic. **Choose 302 if analytics or expiry matter**, and say that you are choosing it for that reason.

Step 4: Data model and storage

```
CREATE TABLE url_mapping (
    code        varchar(10) PRIMARY KEY,
    long_url    text        NOT NULL,
    user_id     text,
    created_at  timestamptz NOT NULL DEFAULT now(),
    expires_at  timestamptz
);
```

- The access pattern is a **key-value lookup by code**, with no joins. That suits a key-value or wide-column store (DynamoDB, Cassandra) that scales horizontally.
- You can also argue that **PostgreSQL with read replicas and a cache** handles this scale at first, because 10,000 reads per second mostly hit the cache. Say which you would start with and **when you would shard** (when data or writes outgrow one primary), and shard by a hash of the code.

Step 5: High-level architecture

```
Client --> Load balancer --> Stateless app servers --> Redis cache
                                    |                       |
                                    |                       v (miss)
                                    +----------------> Database (replicated)
                                    |
                                    +--> Kafka (click events) --> Analytics
consumers
```

Read path (redirect)

1. The app looks up the code in **Redis** (cache-aside).
2. On a hit, return the redirect immediately.
3. On a miss, read the database, store the result in Redis with a **TTL**, and redirect.
4. For unknown codes, cache a short-lived **negative entry**, so scanners cannot hammer the database.

5. Publish a **click event to Kafka asynchronously**, so analytics never slows the redirect or fails it.

Write path (create)

1. Validate the URL (allowed scheme, length, not on a blocklist) and apply **rate limits** per user and IP.
2. Generate the code (next step), or check that the custom alias is free.
3. Insert into the database. A **unique constraint** on the code protects against races.
4. Return the short URL.

Step 6: Deep dive, generating the code

Option A: hash the long URL

- Take an MD5 or SHA hash, encode it in base62, and keep the first 7 characters.
- Simple and stateless, but **collisions** can happen. On a collision, add a salt and retry, which needs a database check on every write.

Option B: counter encoded in base62 (a strong default)

- Each new link takes the next number from a counter and encodes it. There are **no collisions**.
- A single counter is a bottleneck and a single point of failure. Fix it by **allocating ranges**. Each app instance reserves a block, for example 1 million ids, from a central source (a database sequence, Redis INCR or ZooKeeper) and hands them out locally. A crashed instance wastes only its unused block, which is harmless.
- Sequential codes are **guessable and enumerable**. If links are private, scramble the number with a reversible permutation or XOR with a secret before encoding.

```
private static final String ALPHABET =
        "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

static String encode(long n) {
    if (n == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (n > 0) {
        sb.append(ALPHABET.charAt((int) (n % 62)));
        n /= 62;
    }
    return sb.reverse().toString();
}
```

Option C: a key generation service

- A separate service pre-generates unique random keys and hands out batches to app servers. No collisions and unpredictable, at the cost of one more component to run and protect.

Option D: random code plus a unique constraint

- Generate a random 7-character code, insert, and **retry on a unique violation**. Simple, and collisions are rare while the table is small, but they become more frequent as it fills.

A good answer: pick one (for example B with range allocation), explain why, and mention the others as alternatives.

Step 7: Scaling, failures and trade-offs

- **Cache failure:** fall back to the database, protected by a circuit breaker and rate limiting (Day 3), so a cold cache does not flatten the database.
- **Hot links:** a viral link creates a hot key. Use a very high cache hit rate, local in-process caching for the hottest codes, and request coalescing so one miss does not trigger thousands of identical database reads (the stampede problem). Add TTL jitter so keys do not all expire together.
- **Database scaling:** read replicas for reads, sharding by code hash for writes and size, and replication across availability zones for durability.
- **Multi-region:** serve redirects from the nearest region with replicated data, and accept a little staleness on new links. Writes can go to one home region.
- **Expiry and cleanup:** check expires_at at read time (return 410), and delete old rows with a background job or time-based partitions.
- **Abuse:** URL shorteners attract phishing and spam. Rate limit, check URLs against safe-browsing lists, allow reporting, and block known bad domains.
- **Analytics:** keep the click stream in Kafka, aggregate in a separate analytics store, and accept that counts are **eventually consistent**.
- **Monitoring:** redirect latency percentiles, cache hit rate, error rate, database load, id range exhaustion and Kafka lag.
- **Security:** validate input, avoid open redirect abuse in your own admin pages, use HTTPS, and require authentication for management endpoints.

Likely follow-up questions

1. How do you guarantee unique codes across many app servers?
2. How do you support custom aliases without breaking the generator?
3. 301 or 302, and why?

4. How do you handle a link that goes viral?
5. How do you shard the database, and what happens when you add a node?
6. How do you delete or update a link when it is cached everywhere?
7. How would you count clicks accurately at this scale?
8. What changes if links must be private or expire after one use?

5. Practice Exercise (45 Minutes)

1. Close this guide. Take a blank page or whiteboard and a timer.
2. Design the URL shortener aloud for **35 minutes**, following the seven steps. Record yourself if you can.
3. Open section 4 and compare. Write down **three things you missed** and **one thing you explained better than the guide**.
4. Repeat tomorrow with a different prompt of similar size, such as a paste bin or an image sharing service, to check that the framework works on something new.

6. Week 3 Mock Interview (30 to 40 Minutes)

Answer aloud in 90 seconds or less each. Score yourself: 2 = complete, 1 = partial, 0 = could not answer.

1. What is a microservice, and when would you not use microservices?
2. How do you decide service boundaries? What is a distributed monolith?
3. What does an API gateway do, and what should it not do?
4. How does service discovery work in Eureka and in Kubernetes?
5. REST vs gRPC vs messaging: how do you choose?
6. How do you set timeouts across a chain of three services?
7. When is retrying safe, and what is retry amplification?
8. Explain the circuit breaker states and the settings that make it trip.
9. Bulkhead, rate limiter and time limiter: what does each protect against?
10. What is a saga? Choreography vs orchestration?
11. What is the dual-write problem, and how does the outbox pattern solve it?
12. How do you make a Kafka consumer idempotent?
13. Explain topics, partitions, consumer groups and offsets. Where is ordering guaranteed?
14. What happens during a consumer group rebalance, and how can it cause duplicates?
15. What is CQRS, and what does it cost you?

16. Describe how you would handle a payment that succeeded when the inventory reservation then failed.

Scoring (maximum 32)

- 26 to 32: strong. Fix the few weak spots.
- 20 to 25: decent. Reread the day guides for each 0 and 1.
- Below 20: spend an extra session on the weak topics before the Week 4 mini project work.

7. Project Checkpoint

Before Week 4 begins, confirm that:

- [ ] The repository is on GitHub and the README explains how to run it.
- [ ] docker compose starts PostgreSQL, Kafka, Redis and all services.
- [ ] Login works and the gateway rejects calls without a valid token.
- [ ] POST /api/orders saves an order and an outbox row in one transaction, and the idempotency key works.
- [ ] You can explain each design decision from Day 6 in under a minute.

If something is unfinished, spend 30 minutes finishing it now. Week 4 builds directly on it.

8. Week 3 Review (Write in Your Notes)

- What was my mock score, and which topics scored 0 or 1?
- Which concept became clear only after I tried it in code (timeouts, a circuit breaker, the outbox, rebalancing)?
- Which DSA problems from this week do I still not trust myself on?
- What was my hardest moment in the project, and what did I learn from it?
- What will I change in my routine for Week 4?

9. Week 4 Preview: Kafka, Redis and Finishing the Project

- **Day 1:** Kafka producers (acks, idempotence, partitioning, ordering). DSA: heaps (Kth Largest Element, Top K Frequent Elements, Merge K Sorted Lists).
- **Day 2:** Kafka consumers (offsets, rebalancing, retry topics, dead letter queue, exactly-once). DSA: Find Median from Data Stream, Task Scheduler.
- **Day 3:** Redis data structures and caching patterns (cache-aside, write-through, TTL, eviction, stampede). DSA: binary search on the answer.

- **Day 4:** Redis rate limiting and distributed locks, Spring Cache with Redis. DSA: tries.
- **Day 5:** project work: wire Kafka events and the Redis cache, and write tests (JUnit, Mockito, Testcontainers).
- **Day 6:** project work: Resilience4j, Actuator and Micrometer metrics, a README with an architecture diagram, push to GitHub.
- **Day 7:** system design: rate limiter, plus a mock interview.

10. End-of-Day Checklist

- [ ] I can recite the seven HLD steps and the time budget for each.
- [ ] I can do a quick estimation of requests per second, storage and cache size.
- [ ] I designed the URL shortener aloud in 35 minutes and compared it with the guide.
- [ ] I can explain at least three ways to generate short codes and their trade-offs.
- [ ] I can explain 301 vs 302 and the cache-aside read path with negative caching.
- [ ] I completed the Week 3 mock and wrote my review notes.
- [ ] My project checkpoint is complete.

Next (Week 4, Day 1): Kafka producers in depth (acks, idempotence, partitioning, ordering), plus the first heap problems.